

DATABASE MANAGEMENT SYSTEM FOR A PUBLIC LIBRARY

CS-504 | Prof. Binqian Yin, Ph.D

Anil Kumar Mureboina

George Mason University

Fairfax, Virginia

amureboi@gmu.edu

ABSTRACT

The public library is an essential institution in contemporary society, providing members with access to knowledge and resources. A comprehensive database management system is essential for managing the enormous collection of library materials. The goal of this project is to create a database system that makes it easier to handle library materials and information about members. To increase data integrity, the system should efficiently and accurately track borrowed items and avoid redundancy. The suggested system would allow library personnel to manage their collection and members to browse, search, and borrow resources conveniently. In the end, the goal of the project is to improve how the library works so that it is easier for people to use and provides better service.

1. INTRODUCTION

This is the final project for CS504 Principles of Data Management and Mining. This project is intended to assess the students' understanding of course-wide concepts, such as database design, implementation, and querying. It will be necessary to develop a functional database system for a public library, an essential institution in modern society that provides members with access to knowledge and resources. The database should ensure that data is stored efficiently and is readily accessible.

2. PROJECT SCENARIO

2.1. OBJECTIVE

The goal of this project is to make a database management system for a public library and put it into action. The system will help the library staff organize their books, magazines, digital media, and other materials. It will also make it easier to get information about library members and to borrow and keep track of library items. The library management system must guarantee data integrity and minimize redundancy for both library staff and library members.

2.2. FEATURES

- **Materials Management:** The system should store and keep track of information about all library materials, such as books, magazines, e-books, and podcasts, including their names, writers, release dates, and genres.
- **Membership Management:** The system should keep and handle information about library members, such as their names, contact information, membership numbers, and borrowing records.
- **Borrowing:** The system should allow members to check out products as well as provide staff members the information they need to manage circulation. A librarian should note the borrow date and expected due date of a material. The material's return date should be revised.
- **Reporting and Analytics:** The system should create reports on library usage, popular materials, and other statistics to help librarians make resource acquisition and management decisions.

3. DATABASE MANAGEMENT SYSTEM

3.1. DATABASE DESIGN

The scope of the project is to design and implement a database management system for a public library that facilitates the efficient administration of library resources and member data. The system will allow library staff to administer their collection, monitor borrowed materials, reduce redundancies, and maintain data integrity.

The entities and relationships in this database schema are as follows:

- 1) **Material:** Represents individual items available in the library, such as books, magazines, e-books, and audiobooks.
Attributes: Material_ID, Title, Publication_Date, Catalog_ID, Genre_ID.
- 2) **Catalog:** Represents a record of library materials with information on their availability and location.
Attributes: Catalog_ID, Name, Location.
- 3) **Genre:** Represents the various genres or categories of library materials.
Attributes: Genre_ID, Name, Description.
- 4) **Borrow:** Represents the borrowing activity of library materials by members.
Attributes: Borrow_ID, Material_ID, Member_ID, Staff_ID, Borrow_Date, Due_Date, Return_Date.
- 5) **Author:** Represents authors who have created library materials.
Attributes: Author_ID, Name, Birth_Date, Nationality.
- 6) **Authorship:** Represents the relationship between authors and the materials they have created.
Attributes: Authorship_ID, Author_ID, Material_ID.
- 7) **Member:** Represents library members who can borrow and reserve materials.
Attributes: Member_ID, Name, Contact_Info, Join_Date.
- 8) **Staff:** Represents library staff who manage library resources and assist members
Attributes: Staff_ID, Name, Contact_Info, Job_Title, Hire_Date.

RELATIONAL DATABASE SCHEMA

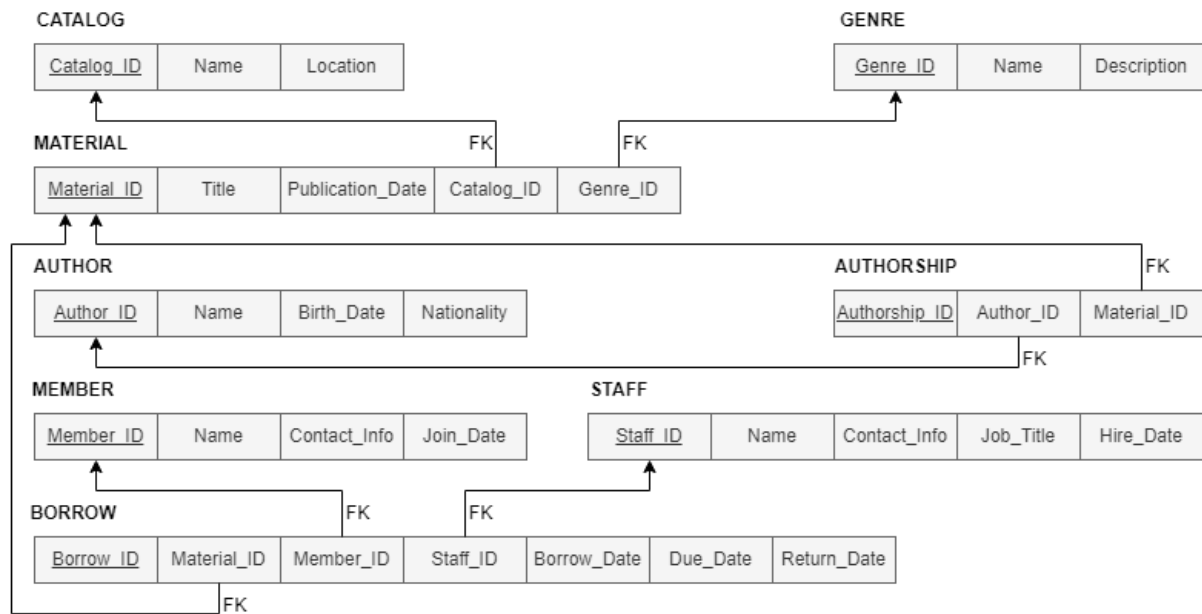


Figure 1: Relational Database Schema

ENTITY-RELATIONSHIP (ER) DIAGRAM

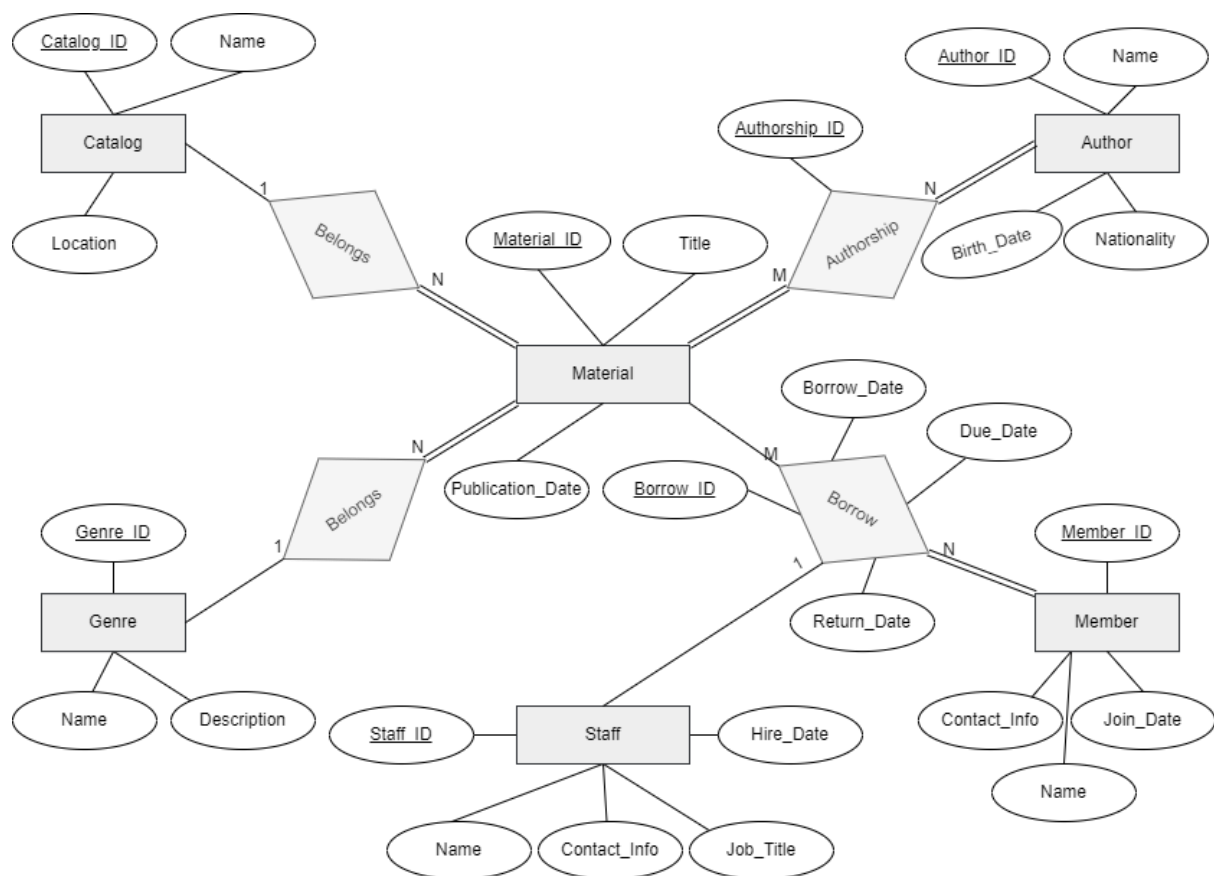


Figure 2: Entity Relationship Diagram

According to the ER diagram, the relationships between the entities are as follows:

- The Catalog table has a one-to-many relationship with the Material table which means the Catalog may contain multiple Materials, but each Material is restricted to a single Catalog. A material record cannot exist without being linked to Catalog record.
- The Genre table has a one-to-many relationship with the Material table which means the Genre may contain multiple Materials, but each Material is restricted to a single Genre. A material record cannot exist without being linked to Genre record.
- The Author table has a many-to-many relationship with the Material table through the Authorship table. Each Material can have multiple Authors, and each Author can be associated with multiple Materials. Both the tables are strictly dependent on each other.
- A Member table has a many-to-many relationship with the Material table. A Member may borrow multiple Materials, and multiple Members may borrow the same Material. This association is represented by the Borrow table. If a material does not exist in the Material table, it cannot be borrowed. Thus, the Material table is a prerequisite for the Borrow table, which is a prerequisite for the Member table.
- A Staff member can manage multiple Borrow transactions, but only one Staff member manages each Borrow transaction.

3.2. DATABASE IMPLEMENTATION

This project's database management system is PostgreSQL, also known as Postgres, a free and open-source relational database management system that emphasizes extensibility and SQL compliance. Moreover, the database schema was implemented using SQL.

Creation of tables:

```
CREATE TABLE Catalog (  
    Catalog_ID serial PRIMARY KEY,  
    Name varchar(255) NOT NULL,  
    Location varchar(255)  
);  
  
CREATE TABLE Genre (  
    Genre_ID serial PRIMARY KEY,  
    Name varchar(255) NOT NULL,  
    Description TEXT  
);  
  
CREATE TABLE Material (  
    Material_ID serial PRIMARY KEY,  
    Title varchar(255) NOT NULL,  
    Publication_Date DATE,  
    Catalog_ID INTEGER NOT NULL,  
    Genre_ID INTEGER NOT NULL,  
    FOREIGN KEY (Catalog_ID) REFERENCES Catalog(Catalog_ID),  
    FOREIGN KEY (Genre_ID) REFERENCES Genre(Genre_ID)  
);
```

```
CREATE TABLE Author (  
    Author_ID serial PRIMARY KEY,  
    Name varchar(255) NOT NULL,  
    Birth_Date DATE,  
    Nationality varchar(255)  
);  
  
CREATE TABLE Authorship (  
    Authorship_ID serial PRIMARY KEY,  
    Author_ID INTEGER NOT NULL,  
    Material_ID INTEGER NOT NULL,  
    FOREIGN KEY (Author_ID) REFERENCES Author(Author_ID),  
    FOREIGN KEY (Material_ID) REFERENCES Material(Material_ID)  
);  
  
CREATE TABLE Member (  
    Member_ID serial PRIMARY KEY,  
    Name varchar(255) NOT NULL,  
    Contact_Info varchar(255) NOT NULL,  
    Join_Date DATE NOT NULL  
);  
  
CREATE TABLE Staff (  
    Staff_ID serial PRIMARY KEY,  
    Name varchar(255) NOT NULL,  
    Contact_Info varchar(255) NOT NULL,  
    Job_Title varchar(255) NOT NULL,  
    Hire_Date DATE NOT NULL  
);  
  
CREATE TABLE Borrow (  
    Borrow_ID serial PRIMARY KEY,  
    Material_ID INTEGER NOT NULL,  
    Member_ID INTEGER NOT NULL,  
    Staff_ID INTEGER NOT NULL,  
    Borrow_Date DATE NOT NULL,  
    Due_Date DATE NOT NULL,  
    Return_Date DATE,  
    FOREIGN KEY (Material_ID) REFERENCES Material(Material_ID),  
    FOREIGN KEY (Member_ID) REFERENCES Member(Member_ID),  
    FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID)  
);
```

Insertion of data:

Insert the sample data into all the tables.

```
INSERT INTO Catalog (Name, Location)
VALUES ('Books', 'A1.1'),
       ('Magazines', 'B2.1'),
       ('E-Books', 'C3.1'),
       ('Audiobooks', 'D4.1'),
       ('Journals', 'E5.1'),
       ('Newspaper', 'F6.1'),
       ('Maps', 'G7.1'),
       ('Novels', 'H8.1'),
       ('Sheet Music', 'I9.1'),
       ('Educational', 'J10.1');
```

```
INSERT INTO Genre (Name, Description)
VALUES ('General Fiction', 'Literary works with a focus on character and plot development, exploring various themes and genres'),
       ('Mystery & Thriller', 'Suspenseful stories centered around crime, investigation, or espionage with an emphasis on solving mysteries'),
       ('Science Fiction & Fantasy', 'Imaginative works that explore alternate realities, futuristic concepts, and magical worlds'),
       ('Horror & Suspense', 'Stories designed to evoke fear, unease, or dread, often featuring supernatural or psychological elements'),
       ('Dystopian & Apocalyptic', 'Depictions of societies in decline or collapse, often exploring themes of political oppression and societal collapse'),
       ('Classics', 'Enduring works of literature that have stood the test of time, often featuring rich language and historical settings'),
       ('Historical Fiction', 'Fictional stories set in the past, often based on real historical events or figures, and blending fiction with historical accuracy'),
       ('Epic Poetry & Mythology', 'Ancient or traditional stories and poems, often featuring heroes, gods, and mythological creatures');
```

```
INSERT INTO Material (Title, Publication_Date, Catalog_ID, Genre_ID)
VALUES ('The Catcher in the Rye', '1951-07-16', 1, 1),
       ('To Kill a Mockingbird', '1960-07-11', 2, 1),
       ('The Da Vinci Code', '2003-04-01', 3, 2),
       ('The Hobbit', '1937-09-21', 4, 3),
       ('The Shining', '1977-01-28', 5, 4),
       ('Pride and Prejudice', '1813-01-28', 1, 1),
       ('The Great Gatsby', '1925-04-10', 2, 1),
       ('Moby Dick', '1851-10-18', 3, 1),
       ('Crime and Punishment', '1866-01-01', 4, 1),
       ('The Hitchhiker's Guide to the Galaxy', '1979-10-12', 5, 3),
       ('1984', '1949-06-08', 1, 5),
       ('Animal Farm', '1945-08-17', 2, 5),
       ('The Haunting of Hill House', '1959-10-17', 3, 4),
       ('Brave New World', '1932-08-01', 4, 5),
       ('The Chronicles of Narnia: The Lion, the Witch and the Wardrobe', '1950-10-16', 5, 3),
       ('The Adventures of Huckleberry Finn', '1884-12-10', 6, 1),
       ('The Catch-22', '1961-10-11', 7, 1),
       ('The Picture of Dorian Gray', '1890-07-01', 8, 1),
       ('The Call of Cthulhu', '1928-02-01', 9, 4),
       ('Harry Potter and the Philosopher's Stone', '1997-06-26', 10, 3),
       ('Frankenstein', '1818-01-01', 6, 4),
       ('A Tale of Two Cities', '1859-04-30', 7, 1),
       ('The Iliad', '1750-01-01', 8, 6),
       ('The Odyssey', '1725-01-01', 9, 6),
       ('The Brothers Karamazov', '1880-01-01', 10, 1),
       ('The Divine Comedy', '1320-01-01', 6, 6),
       ('The Grapes of Wrath', '1939-04-14', 7, 1),
```

```
INSERT INTO Author (Name, Birth_Date, Nationality)
VALUES ('Jane Austen', '1775-12-16', 'British'),
('Ernest Hemingway', '1899-07-21', 'American'),
('George Orwell', '1903-06-25', 'British'),
('Scott Fitzgerald', '1896-09-24', 'American'),
('J.K. Rowling', '1965-07-31', 'British'),
('Mark Twain', '1835-11-30', 'American'),
('Leo Tolstoy', '1828-09-09', 'Russian'),
('Virginia Woolf', '1882-01-25', 'British'),
('Gabriel Márquez', '1927-03-06', 'Colombian'),
('Charles Dickens', '1812-02-07', 'British'),
('Harper Lee', '1926-04-28', 'American'),
('Oscar Wilde', '1854-10-16', 'Irish'),
('William Shakespeare', '1564-04-26', 'British'),
('Franz Kafka', '1883-07-03', 'Czech'),
('James Joyce', '1882-02-02', 'Irish'),
('J.R.R. Tolkien', '1892-01-03', 'British'),
('Emily Brontë', '1818-07-30', 'British'),
('Toni Morrison', '1931-02-18', 'American'),
('Fyodor Dostoevsky', '1821-11-11', 'Russian'),
('Lucas Piki', '1847-10-16', 'British');
```

```
INSERT INTO Authorship (Author_ID, Material_ID)
VALUES (1, 1), (2, 2),
(3, 3), (4, 4),
(5, 5), (6, 6),
(7, 7), (8, 8),
(9, 9), (10, 10),
(11, 11), (12, 12),
(13, 13), (14, 14),
(15, 15), (16, 16),
(17, 17), (18, 18),
(19, 19), (20, 20),
(1, 21), (2, 22),
(3, 23), (4, 24),
(5, 25), (6, 26),
(7, 27), (8, 28),
(19, 28), (9, 29),
(10, 30), (8, 30), (2, 29);
```



```
INSERT INTO Member (Name, Contact_Info, Join_Date)
VALUES ('Alice Johnson', 'alice.johnson@email.com', '2018-01-10'),
('Bob Smith', 'bob.smith@email.com', '2018-03-15'),
('Carol Brown', 'carol.brown@email.com', '2018-06-20'),
('David Williams', 'david.williams@email.com', '2018-09-18'),
('Emily Miller', 'emily.miller@email.com', '2019-02-12'),
('Frank Davis', 'frank.davis@email.com', '2019-05-25'),
('Grace Wilson', 'grace.wilson@email.com', '2019-08-15'),
('Harry Garcia', 'harry.garcia@email.com', '2019-11-27'),
('Isla Thomas', 'isla.thomas@email.com', '2020-03-04'),
('Jack Martinez', 'jack.martinez@email.com', '2020-07-01'),
('Kate Anderson', 'kate.anderson@email.com', '2020-09-30'),
('Luke Jackson', 'luke.jackson@email.com', '2021-01-18'),
('Mia White', 'mia.white@email.com', '2021-04-27'),
('Noah Harris', 'noah.harris@email.com', '2021-07-13'),
('Olivia Clark', 'olivia.clark@email.com', '2021-10-05'),
('Peter Lewis', 'peter.lewis@email.com', '2021-12-01'),
('Quinn Hall', 'quinn.hall@email.com', '2022-02-28'),
('Rachel Young', 'rachel.young@email.com', '2022-06-17'),
('Sam Walker', 'sam.walker@email.com', '2022-09-25'),
('Tiffany Allen', 'tiffany.allen@email.com', '2022-12-10');

INSERT INTO Staff (Name, Contact_Info, Job_Title, Hire_Date)
VALUES ('Amy Green', 'amy.green@email.com', 'Librarian', '2017-06-01'),
('Brian Taylor', 'brian.taylor@email.com', 'Library Assistant', '2018-11-15'),
('Christine King', 'chris.king@email.com', 'Library Assistant', '2019-05-20'),
('Daniel Wright', 'dan.wright@email.com', 'Library Technician', '2020-02-01');

INSERT INTO Borrow (Material_ID, Member_ID, Staff_ID, Borrow_Date, Due_Date, Return_Date)
VALUES (1, 1, 1, '2018-09-12', '2018-10-03', '2018-09-30'),
(2, 2, 1, '2018-10-15', '2018-11-05', '2018-10-29'),
(3, 3, 1, '2018-12-20', '2019-01-10', '2019-01-08'),
(4, 4, 1, '2019-03-11', '2019-04-01', '2019-03-27'),
(5, 5, 1, '2019-04-20', '2019-05-11', '2019-05-05'),
(6, 6, 1, '2019-07-05', '2019-07-26', '2019-07-21'),
(7, 7, 1, '2019-09-10', '2019-10-01', '2019-09-25'),
(8, 8, 1, '2019-11-08', '2019-11-29', '2019-11-20'),
(9, 9, 1, '2020-01-15', '2020-02-05', '2020-02-03'),
(10, 10, 1, '2020-03-12', '2020-04-02', '2020-03-28'),
(1, 11, 2, '2020-05-14', '2020-06-04', '2020-05-28'),
(2, 12, 2, '2020-07-21', '2020-08-11', '2020-08-02'),
(3, 13, 2, '2020-09-25', '2020-10-16', '2020-10-15'),
(4, 1, 2, '2020-11-08', '2020-11-29', '2020-11-24'),
(5, 2, 2, '2021-01-03', '2021-01-24', '2021-01-19'),
(6, 3, 2, '2021-02-18', '2021-03-11', '2021-03-12'),
(17, 4, 2, '2021-04-27', '2021-05-18', '2021-05-20'),
(18, 5, 2, '2021-06-13', '2021-07-04', '2021-06-28'),
(19, 6, 2, '2021-08-15', '2021-09-05', '2021-09-03'),
(20, 7, 2, '2021-10-21', '2021-11-11', '2021-11-05'),
(21, 1, 3, '2021-11-29', '2021-12-20', NULL),
(22, 2, 3, '2022-01-10', '2022-01-31', '2022-01-25'),
(23, 3, 3, '2022-02-07', '2022-02-28', '2022-02-23'),
(24, 4, 3, '2022-03-11', '2022-04-01', '2022-03-28'),
(25, 5, 3, '2022-04-28', '2022-05-19', '2022-05-18'),
(26, 6, 3, '2022-06-22', '2022-07-13', '2022-07-08'),
(27, 7, 3, '2022-08-04', '2022-08-25', '2022-08-23');
```


4. QUERYING AND MANIPULATION

In this project, I developed a set of SQL queries to perform common tasks such as searching, updating, inserting, and deleting records. I also demonstrated the use of advanced querying techniques, including joins, aggregation, and subqueries. The queries listed below are some examples.

1) Which materials are currently available in the library?

Query 1:

```
SELECT m.title FROM Material AS m
WHERE NOT EXISTS
(SELECT * FROM Borrow AS b
WHERE m.material_id = b.material_id AND b.return_date IS NULL);
```

Query 2:

```
SELECT m1.title FROM Material AS m1,
(SELECT material_id FROM Material
EXCEPT
SELECT material_id FROM Borrow WHERE return_date IS NULL) AS m2
WHERE m1.material_id = m2.material_id
ORDER BY m1.material_id;
```

Output:

	title character varying (100)	
1	The Da Vinci Code	
2	1984	
3	Animal Farm	
4	The Haunting of Hill House	
5	Brave New World	
6	The Chronicles of Narnia: The Lion, the Witch and the Wardro...	
7	The Adventures of Huckleberry Finn	
8	The Catch-22	
9	The Picture of Dorian Gray	
10	The Call of Cthulhu	
11	Harry Potter and the Philosopher's Stone	
12	A Tale of Two Cities	
13	The Iliad	
14	The Odyssey	
15	The Brothers Karamazov	
16	The Divine Comedy	
17	The Grapes of Wrath	
18	The Old Man and the Sea	
19	The Count of Monte Cristo	
20	A Midsummer Night's Dream	
21	The Tricky Book	

2) Which materials are currently overdue? Suppose today is 04/01/2023 and show the borrow date and due date of each material.

Query 1:

```
SELECT m.title, b.borrow_date, b.due_date FROM Borrow AS b, Material AS m
WHERE b.material_id = m.material_id
AND b.due_date < '2023-04-01'
AND b.return_date IS NULL;
```

Query 2:

```
SELECT m.title, b.borrow_date, b.due_date FROM Borrow AS b
INNER JOIN Material AS m
ON b.material_id = m.material_id
AND b.due_date < '2023-04-01'
AND b.return_date IS NULL;
```

Output:

	title character varying (100) 🔒	borrow_date date 🔒	due_date date 🔒
1	The Catcher in the Rye	2022-12-28	2023-01-18
2	To Kill a Mockingbird	2023-01-23	2023-02-13
3	The Hobbit	2023-03-01	2023-03-22
4	The Shining	2023-03-10	2023-03-31
5	Frankenstein	2021-11-29	2021-12-20

3) What are the top 10 most borrowed materials in the library? Show the title of each material and order them based on their available counts.

Query 1:

```
SELECT m.title FROM Material AS m, Borrow AS b
WHERE b.material_id = m.material_id
GROUP BY m.title
ORDER BY COUNT(b.material_id) DESC
LIMIT 10;
```

Query 2:

```
SELECT m.title FROM Material AS m
INNER JOIN Borrow AS b
ON b.material_id = m.material_id
GROUP BY m.title
ORDER BY COUNT(b.material_id) DESC
LIMIT 10;
```

Output:

	title character varying (100) 
1	The Catcher in the Rye
2	Pride and Prejudice
3	The Da Vinci Code
4	The Hobbit
5	To Kill a Mockingbird
6	The Shining
7	The Great Gatsby
8	Moby Dick
9	Crime and Punishment
10	The Hitchhiker's Guide to the Galaxy

4) How many books has the author Lucas Piki written?**Query:**

```
SELECT COUNT(*) AS Number_of_Books FROM
(SELECT a1.material_id FROM Authorship AS a1, Author AS a2, Material AS m
WHERE a1.author_id = a2.author_id AND
      a2.name = 'Lucas Piki' AND
      a1.material_id = m.material_id AND
      m.catalog_id = (SELECT catalog_id FROM Catalog WHERE name = 'Books')) AS result;
```

Output:

	number_of_books bigint 
1	0

5) How many books were written by two or more authors?**Query:**

```
SELECT COUNT(*) AS number_of_books FROM
(SELECT a.material_id FROM Authorship AS a, Material AS m
WHERE a.material_id = m.material_id AND
      m.catalog_id = (SELECT catalog_id FROM Catalog WHERE name = 'Books')
GROUP BY a.material_id
HAVING COUNT(a.author_id) >= 2) AS result;
```

Output:

	number_of_books bigint 
1	0

6) What are the most popular genres in the library?

The popular genres based on the materials available in the library.

Query 1:

```
SELECT g.name FROM Genre as g, Material as m
WHERE g.genre_id = m.genre_id
GROUP BY g.name
ORDER BY COUNT(g.genre_id) DESC;
```

Query 2:

```
SELECT g.name FROM Genre as g
INNER JOIN Material as m
ON g.genre_id = m.genre_id
GROUP BY g.name
ORDER BY COUNT(g.genre_id) DESC;
```

Output:

	name character varying (50) 
1	General Fiction
2	Science Fiction & Fantasy
3	Horror & Suspense
4	Dystopian & Apocalyptic
5	Classics
6	Historical Fiction
7	Mystery & Thriller

The popular genres based on the materials borrowed by the people.

Query 1:

```
SELECT g.name FROM Genre as g, Material as m, Borrow as b
WHERE g.genre_id = m.genre_id AND m.material_id = b.material_id
GROUP BY g.name
ORDER BY COUNT(g.genre_id) DESC;
```

Query 2:

```
SELECT g1.name FROM Genre AS g1,
(SELECT genre_id FROM Material NATURAL JOIN Borrow) AS g2
WHERE g1.genre_id = g2.genre_id
GROUP BY g1.genre_id
ORDER BY COUNT(g1.genre_id) DESC;
```

Output:

	name character varying (50)
1	General Fiction
2	Science Fiction & Fantasy
3	Horror & Suspense
4	Classics
5	Mystery & Thriller
6	Historical Fiction

7) How many materials have been borrowed from 09/2020-10/2020?**Query:**

```
SELECT COUNT(*) AS number_of_materials FROM Borrow
WHERE borrow_date >= '2020-09-01' AND
      borrow_date < '2020-11-01';
```

Output:

	number_of_materials bigint
1	1

8) How do you update the “Harry Potter and the Philosopher’s Stone” when it is returned on 04/01/2023?**Query:**

```
-- Update Operation
UPDATE Borrow
SET return_date = '2023-04-01'
WHERE material_id = (SELECT material_id FROM Material
                    WHERE title = 'Harry Potter and the Philosopher''s Stone');

-- Check the updated table
SELECT * FROM Borrow
```

Output:

	borrow_id [PK] integer	material_id integer	member_id integer	staff_id integer	borrow_date date	due_date date	return_date date
33	34	4	11	4	2023-03-01	2023-03-22	[null]
34	35	5	12	4	2023-03-10	2023-03-31	[null]
35	36	6	13	4	2023-03-15	2023-04-05	[null]
36	37	7	17	4	2023-03-25	2023-04-15	[null]
37	38	8	8	4	2023-03-30	2023-04-20	[null]
38	39	9	9	4	2023-03-26	2023-04-16	[null]
39	40	10	20	4	2023-03-28	2023-04-18	[null]
40	20	20	7	2	2021-10-21	2021-11-11	2023-04-01

9) How do you delete the member Emily Miller and all her related records from the database?

First, delete the current foreign key constraint for member_id. Then, add the ON DELETE CASCADE option to the member_id foreign key constraint. This will make sure that all records in the Borrow table that are connected to the member who has been removed will be deleted immediately. Then, execute the SQL query to delete the Emily Miller from Member table, that will automatically delete her record from the Borrow table as well.

Query:

```
ALTER TABLE Borrow
DROP CONSTRAINT borrow_member_id_fkey;

ALTER TABLE Borrow
ADD CONSTRAINT borrow_member_id_fkey
FOREIGN KEY (Member_ID)
REFERENCES Member(Member_ID)
ON DELETE CASCADE;

DELETE FROM Member WHERE name = 'Emily Miller';
```

10) How do you add the following material to the database?

Title: New book

Date: 2020-08-01

Catalog: E-Books

Genre: Mystery & Thriller

Author: Lucas Pipi

Query:

```
INSERT INTO Material (Title, Publication_Date, Catalog_ID, Genre_ID)
VALUES
('New book',
 '2020-08-01',
 (SELECT catalog_id FROM Catalog WHERE name = 'E-Books'),
 (SELECT genre_id FROM Genre WHERE name = 'Mystery & Thriller'));

INSERT INTO Author (Name)
VALUES ('Lucas Pipi');

INSERT INTO Authorship (Author_ID, Material_ID)
VALUES ((SELECT Author_ID FROM Author WHERE name = 'Lucas Pipi'),
 (SELECT Material_ID FROM Material WHERE title = 'New book'));
```

-- Check the updated Material table

```
SELECT * FROM Material
```

-- Check the updated Author table

```
SELECT * FROM Author
```

-- Check the updated Authorship table

```
SELECT * FROM Authorship
```

Output:

Material table:

	material_id [PK] integer	title character varying (100)	publication_date date	catalog_id integer	genre_id integer
25	25	The Brothers Karamazov	1880-01-01	10	1
26	26	The Divine Comedy	1320-01-01	6	6
27	27	The Grapes of Wrath	1939-04-14	7	1
28	28	The Old Man and the Sea	1952-09-01	8	1
29	29	The Count of Monte Cristo	1844-01-01	9	1
30	30	A Midsummer Night's Dream	1596-01-01	10	7
31	31	The Tricky Book	1888-01-01	10	7
32	32	New book	2020-08-01	3	2

Author table:

	author_id [PK] integer	name character varying (20)	birth_date date	nationality character varying (20)
14	14	Franz Kafka	1883-07-03	Czech
15	15	James Joyce	1882-02-02	Irish
16	16	J.R.R. Tolkien	1892-01-03	British
17	17	Emily Brontë	1818-07-30	British
18	18	Toni Morrison	1931-02-18	American
19	19	Fyodor Dostoevsky	1821-11-11	Russian
20	20	Lucas Piki	1847-10-16	British
21	21	Lucas Pipi	[null]	[null]

Authorship table:

	authorship_id [PK] integer	author_id integer	material_id integer
27	27	7	27
28	28	8	28
29	29	19	28
30	30	9	29
31	31	10	30
32	32	8	30
33	33	2	29
34	34	21	32

5. FURTHER RESEARCH

We can extend the existing database system to incorporate new features, such as alerting staff members about overdue materials and determining the membership's active status based on the member's overdue tally. The SQL statements and explanation for implementing these new features are provided below.

1) Alert staff about overdue materials on a daily-basis?

To add this feature, we might have to make a new table called "AlertStaff" and put the information of the staff, the material, the member, and the due date of the overdue materials in it. This would alert the staff that the material is late.

Here's the SQL statement to create a new table "AlertStaff":

```
CREATE TABLE AlertStaff (
    Alert_ID serial PRIMARY KEY,
    Staff_ID int NOT NULL,
    Material_ID int NOT NULL,
    Member_ID int NOT NULL,
    Due_Date DATE NOT NULL,
    Alert_Date DATE NOT NULL DEFAULT CURRENT_DATE,
    FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID)
    FOREIGN KEY (Material_ID) REFERENCES Material(Material_ID),
    FOREIGN KEY (Member_ID) REFERENCES Member(Member_ID)
);
```

Then, we will create a trigger that determines if a new borrowing record with a due date less than the current date and no return date has been added. If this is true, the "AlertStaff" table is updated with the staff id, material id, member id, and due date of the corresponding material. Using the records in the "AlertStaff" table, we can then notify the staff members. After each insert into the Borrow table, the function is automatically triggered.

Here's the SQL statement to create a trigger:

```
CREATE FUNCTION overdue_materials() RETURNS TRIGGER AS
BEGIN
    IF NEW.due_date < CURRENT_DATE AND NEW.return_date IS NULL THEN
        INSERT INTO AlertStaff (Staff_ID, Material_ID, Member_ID, Due_Date)
        VALUES (NEW.Staff_ID, NEW.Material_ID, NEW.Member_ID, NEW.Due_Date);
    END IF;
    RETURN NULL;
END;

CREATE TRIGGER alert_staff_trigger
AFTER INSERT ON Borrow
FOR EACH ROW
EXECUTE FUNCTION overdue_materials();
```

2) Automatically deactivate the membership based on the member's overdue occurrence (>= three times). And reactivate the membership once the member pays the overdue fee.

To implement this functionality, we may add a new column named "Overdue_Count" to the Member table to monitor the number of times a member has not paid. materials. If a member has overdue materials, we can increase their Overdue_Count by 1 when they borrow a material. We can decrease a member's Overdue_Count by 1 when they return a material.

Here's the SQL statement to add the "Overdue_Count" field to the "Member" table:

```
ALTER TABLE Member ADD COLUMN Overdue_Count int NOT NULL DEFAULT 0;
```

Then, we can set up a trigger that checks for overdue materials after inserting or updating the "Borrow" or "Member" table. The trigger sets the member's "Membership_Status" to "Inactive" if they have three or more overdue materials. The trigger sets the member's "Membership_Status" to "Active" if they have less than three overdue materials or zero.

Here's the SQL statement to create a trigger:

```
CREATE FUNCTION membership() RETURNS TRIGGER AS
BEGIN
    IF NEW.Overdue_Count >= 3 THEN
        UPDATE Member SET Membership_Status = 'Inactive' WHERE Member_ID = NEW.Member_ID;
    ELSE
        IF NEW.Membership_Status = 'Inactive' THEN
            UPDATE Member SET Membership_Status = 'Active' WHERE Member_ID = NEW.Member_ID;
        END IF;
    END IF;
    RETURN NULL;
END;

CREATE TRIGGER membership_trigger
AFTER INSERT OR UPDATE OF Return_Date, Overdue_Count ON Borrow
FOR EACH ROW
EXECUTE FUNCTION membership();
```

Note: The above SQL queries to create triggers are prototypes and are not executed.

6. CONCLUSION:

In conclusion, developing a public library database system has been a challenging and gratifying project that allowed me to apply course principles and skills. By successfully building the database system, I demonstrated my ability to design, implement, and query a database to store and handle data effectively. By simplifying data administration and providing a central database for library employees and members, the technology will improve library operations. This project has taught me a lot about data management and mining, including the necessity of database architecture and execution.